

• CURSO TÉCNICO EM INFORMÁTICA

SENACGAMES

GUIA DEFINITIVO DO PROJETO

Um passeio completo pelos bastidores de uma plataforma de distribuição de jogos construída do zero: arquitetura em camadas, segurança, banco de dados e o caminho de cada requisição — do clique do usuário até o SQL Server.

ASP.NET Core

Entity Framework Core

SQL Server

MVC / Razor

ASP.NET Identity

REST API

JWT

SUMÁRIO

FUNDAMENTOS

01 O que é o SenacGames

02 Arquitetura do Projeto

03 Fluxo da Aplicação

CAMADA DE APRESENTAÇÃO

04 ASP.NET MVC

DADOS & PERSISTÊNCIA

05 Entity Framework Core

06 SQL Server

SEGURANÇA

07 ASP.NET Identity

API & PADRÕES DE PROJETO

08 API REST

09 Repository Pattern

10 Service Pattern

11 DTOs

12 AutoMapper

FLUXOS NA PRÁTICA

13 Fluxo de Login

14 Fluxo do Cadastro de Games

ORGANIZAÇÃO DO CÓDIGO

15 Estrutura das Pastas

16 Migrations

17 Boas Práticas

PARA PRATICAR

18 Perguntas Frequentes

19 Glossário Técnico

Bem-vindo ao **SenacGames**!

Você já imaginou como plataformas de distribuição de jogos, como a Steam ou a Epic Games, são desenvolvidas por trás das cortinas? O SenacGames é um projeto educacional projetado exatamente para responder a essa pergunta.

O objetivo do projeto é ensinar você, aluno do Curso Técnico em Informática, a construir uma plataforma web moderna, robusta e segura, utilizando os padrões mais exigidos pelo mercado de trabalho corporativo.



OBJETIVO DO PROJETO

Demonstrar o desenvolvimento de uma aplicação web completa (Full Stack) com controle de acesso, painéis de administração, APIs e consumo de dados utilizando boas práticas de arquitetura.

Tecnologias Utilizadas

O ecossistema do SenacGames foi cuidadosamente escolhido para cobrir todas as frentes de uma aplicação corporativa C#. Eis o nosso cinto de utilidades:

- **Linguagem Principal:** C# (C-Sharp)
- **Framework:** .NET Core (ASP.NET Core para Web e API)
- **Banco de Dados:** Microsoft SQL Server
- **ORM (Object-Relational Mapper):** Entity Framework Core
- **Front-end / UI:** ASP.NET Core MVC com Razor Pages, Bootstrap 5 e CSS Customizado
- **Autenticação e Segurança:** ASP.NET Core Identity e JWT (JSON Web Tokens)
- **Arquitetura:** Onion Architecture (Arquitetura em Cebola) com separação em múltiplas camadas

Arquitetura Escolhida

Ao invés de misturar as telas (HTML), as regras do negócio e o acesso ao banco de dados num único local (o que chamamos de *Código Espaguete*), o SenacGames utiliza a **Arquitetura em Camadas**.

Benefícios dessa arquitetura

01

Manutenibilidade

Se você precisar trocar a cor de um botão, não corre o risco de quebrar o login.

02

Reaproveitamento

A API pode ser usada tanto pelo painel Web quanto pelo aplicativo Desktop sem reescrever código.

03

Trabalho em Equipe

Enquanto um aluno faz o Banco de Dados, outro faz o Front-end sem que um bloqueie o outro.

A grande "mágica" do SenacGames está na sua divisão estrutural.



PENSE COMO UM RESTAURANTE

Há o salão onde os clientes comem (Interface), os garçons (API/Controllers), o chef de cozinha (Service/Regras de negócio) e a despensa (Banco de Dados).

Aqui estão as camadas do projeto:

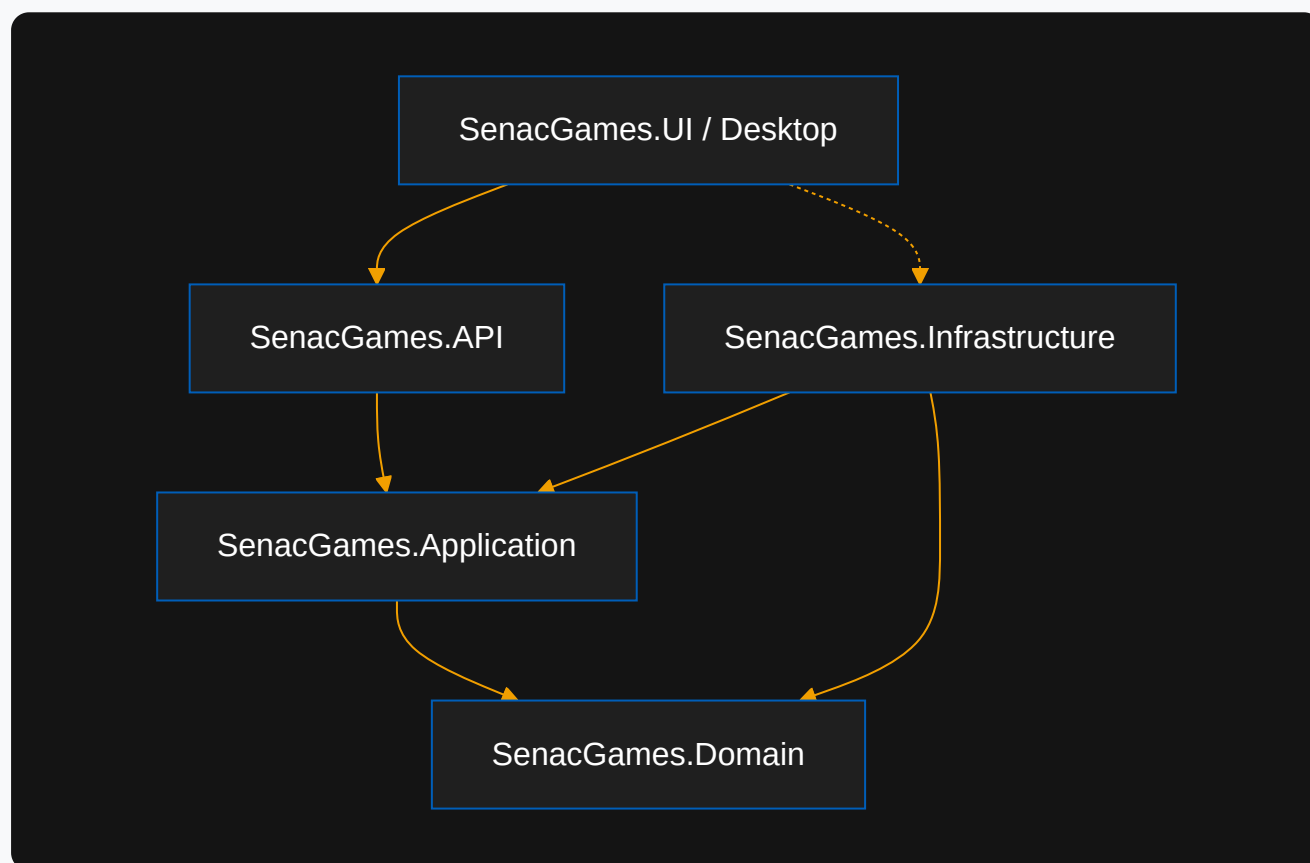


Fig. 2.1 – Dependência entre as camadas do SenacGames

CAMADA 1

SenacGames.Domain — O Coração

Finalidade: contém as Entidades (Classes) que representam as tabelas do banco de dados (ex: `Game.cs`, `Category.cs`).

Responsabilidades: representar o domínio do negócio.

Quando utilizar: quando você precisar criar uma nova tabela, você criará uma classe aqui.

SenacGames.Infrastructure — A Despensa

Finalidade: é a única camada que "conversa" de verdade com o SQL Server.

Responsabilidades: possui o `DbContext` do Entity Framework, as `Migrations` (histórico de alterações do banco) e as implementações de `Repository`.

Exemplos: aqui fica o código que faz os "INSERTs" e "SELECTs" reais.

SenacGames.Application — O Chef de Cozinha

Finalidade: contém as Regras de Negócio e os DTOs.

Responsabilidades: processar os dados. Por exemplo: "Um usuário só pode favoritar 10 games" — é nesta camada que essa regra vive (Service Pattern).

Quando utilizar: toda lógica complexa, validação ou orquestração deve estar aqui.

SenacGames.API — O Garçom

Finalidade: é uma porta de entrada para aplicações externas.

Responsabilidades: fornecer rotas (Endpoints) como `/api/games` que retornam JSON.

Quando utilizar: quando um celular ou um programa Desktop quiser consultar a lista de games, ele chamará a API.

SenacGames.UI — O Salão de Clientes

Finalidade: é a Interface do Usuário (Web) construída em ASP.NET MVC.

Responsabilidades: exibir o visual, carregar o CSS/Imagens (`wwwroot`) e desenhar as telas para o usuário final utilizando Razor (`.cshtml`).

SenacGames.Desktop — O Balcão Gerencial

Finalidade: um aplicativo Windows Forms para a administração do sistema.

Responsabilidades: fornecer telas nativas de Windows para gerenciar o catálogo consumindo a nossa API.

03 FUNDAMENTOS

FLUXO DA APLICAÇÃO

Entender como um "clique" viaja pela aplicação é o segredo para se tornar um programador completo.

Veja o caminho de uma requisição típica quando o cliente quer ver os Detalhes de um Game na Loja Web:

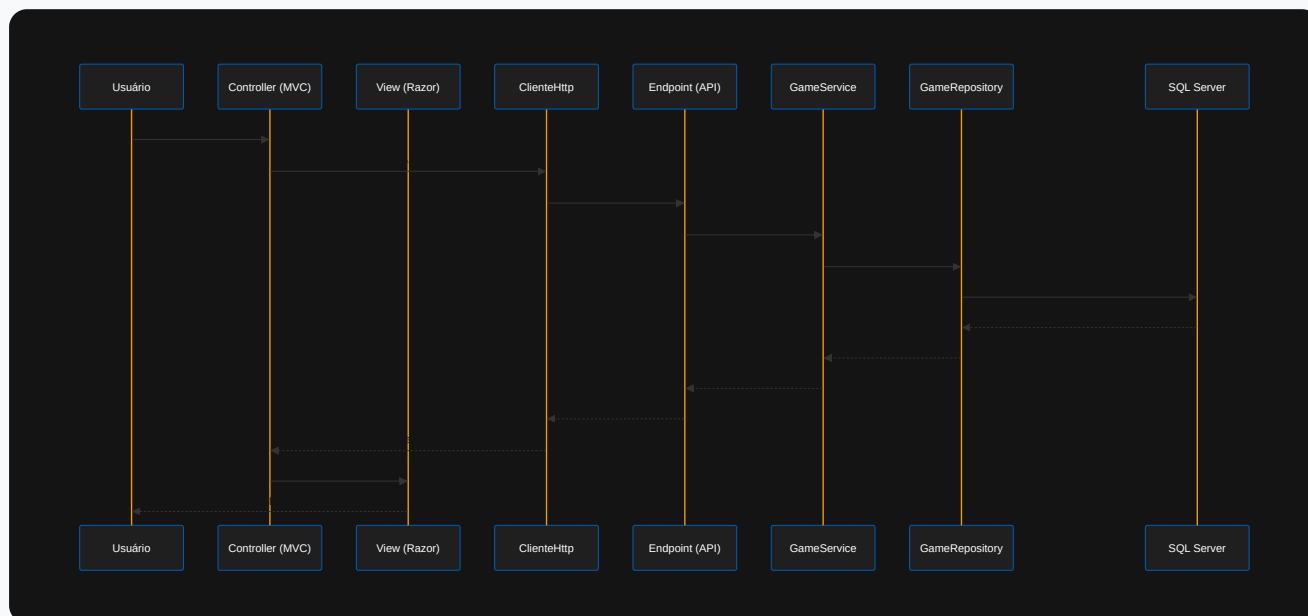


Fig. 3.1 – Diagrama de sequência: exibindo os detalhes de um game

Passo a passo explicado

- 1 O Clique:** o usuário, no navegador, clica na capa de um game. O link vai para `GamesController.cs` no painel MVC.
- 2 A Comunicação:** o MVC percebe que precisa dos dados do game, mas **ele não acessa o banco diretamente**. Ele pede para o serviço de API chamar a API.
- 3 A API:** o `GamesController.cs` na API recebe o pedido `GET /api/games/5`. Ele repassa a tarefa para a camada de Aplicação (`GameService.cs`).
- 4 Regras de Negócio:** o `Service` verifica se há alguma regra antes de entregar os dados.
- 5 O Banco de Dados:** o `GameRepository` utiliza o Entity Framework para executar uma instrução SQL no banco.
- 6 A Volta:** o banco retorna os dados. Esses dados passam pelo Repository, Service, API, convertem para JSON, chegam no MVC, que converte em HTML através da View e devolve para a tela do usuário.

Esse caminho pode parecer longo, mas é o que garante que seu projeto não se torne um caos à medida que cresce!

04 CAMADA DE APRESENTAÇÃO

ASP.NET MVC

O ASP.NET Core MVC é a tecnologia responsável por criar a interface visual (a "cara") do SenacGames na Web. MVC significa **Model-View-Controller**.

1. Model (O Dado)

No MVC, o *Model* representa a estrutura dos dados que vão para a tela. No SenacGames, usamos o padrão **ViewModel**, que é uma classe "ajustada" só com o que a tela precisa.

Exemplo: a tela de detalhes não precisa saber a senha do usuário que cadastrou o game, ela só precisa da classe `GameViewModel`.

2. View (A Tela)

A View é o HTML. No ASP.NET, usamos um motor chamado **Razor** (`.cshtml`). O Razor permite misturar C# com HTML! Veja um exemplo real do projeto (`Details.cshtml`):

```
@model SenacGames.UI.ViewModels.GameViewModel

<h1>@Model.Title</h1>
<p>@Model.Description</p>
```

O `@` indica que o que vem depois é código C#.

3. Controller (O Maestro)

O Controller é a classe C# (`Controller`) que recebe o clique do usuário, pede os dados e decide qual View mostrar.

```
public async Task<IActionResult> Details(int id)
{
    var game = await _gameService.GetGameAsync(id);
    return View(game); // Manda o game para a tela
}
```

Conceitos importantes na MVC do SenacGames

- **Model Binding:** é a mágica que pega o que o usuário digitou em `<input name="Title">` e converte automaticamente para um objeto `GameViewModel` no C#.
- **Partial View:** pedaços de HTML reutilizáveis.
- **Layouts:** o `_Layout.cshtml` é a "forma de bolo". Ele contém o cabeçalho e rodapé que se repetem em todas as páginas, para você não ter que copiar e colar o menu em cada tela nova.

- **Tag Helpers:** são extensões do ASP.NET para facilitar a escrita de HTML. Exemplo: `<a asp-controller="Home" asp-action="Index">` vira um link para a Home sem você precisar saber a URL exata.

O Entity Framework (EF) Core é o nosso ORM.



ANALOGIA

O banco de dados (SQL) só entende tabelas e colunas. O C# só entende Classes e Propriedades. O EF Core é o "tradutor simultâneo" que pega suas classes em C# e transforma magicamente em comandos SQL.

Componentes chave

- **Entities:** suas classes de domínio (ex: `Game.cs`). Cada classe vira uma tabela. Cada propriedade vira uma coluna.
- **DbContext:** o `SenacGamesDbContext.cs` é o coração do EF. Ele gerencia a conexão com o banco e diz quais entidades vão virar tabelas.
- **DbSet:** dentro do DbContext, temos `public DbSet<Game> Games { get; set; }`. Isso significa "Crie uma tabela chamada Games com base na classe Game".

Trabalhando com o banco de dados

- 1 **LINQ:** em vez de escrever `SELECT * FROM Games WHERE Title = 'Cyberpunk 2077'`, nós escrevemos em C# puro:

```
var games = _context.Games.Where(g => g.Title == "Cyberpunk 2077").ToList();
```

- 2 **Migrations:** toda vez que você altera uma classe (ex: adiciona `public int ReleaseYear { get; set; }`), você usa `Add-Migration` e depois `Update-Database`. O EF cria o script SQL sozinho e altera o banco. Adeus criação manual de tabelas!
- 3 **Tracking vs NoTracking:** quando você puxa um dado para editar, o EF fica "vigiando" (Tracking). Se você mudar uma propriedade, ele percebe e salva no banco. Se for só para exibir na tela, usamos `.AsNoTracking()`, muito mais rápido porque o EF desativa esse "olheiro".
- 4 **Relacionamentos:** para criar uma Chave Estrangeira, basta colocar um objeto dentro do outro. O EF entende que um `Game` tem uma `Category` e cria as FKs no SQL automaticamente.

06 DADOS & PERSISTÊNCIA

SQL SERVER

Mesmo usando o Entity Framework Core, o banco de dados real por baixo dos panos é o robusto **Microsoft SQL Server**.

O papel do SQL no projeto

O EF gera as tabelas, as restrições (Constraints) e os relacionamentos, mas o SQL Server é quem armazena e processa isso em disco de forma otimizada.

Onde o SQL entra em cena?

- **Tabelas (Tables):** o EF transforma a classe `Game` na tabela `Games`.
- **Primary Keys (PK):** a propriedade `Id` nas suas classes automaticamente vira a Chave Primária, garantindo a unicidade de cada registro.
- **Foreign Keys (FK):** a propriedade `CategoryId` vira a Chave Estrangeira.
- **Índices:** o EF Core pode criar índices (para buscas mais rápidas) no banco SQL. No SenacGames, criamos índices em campos muito buscados, como o e-mail de usuário, para otimizar os acessos!



DIFERENÇA ENTRE EF E SQL

O EF é a ferramenta de programação. O SQL Server é o motor de armazenamento. Embora o EF facilite o trabalho, você deve entender de SQL para saber se as consultas geradas estão pesadas, se as chaves estrangeiras foram aplicadas corretamente e como otimizar o banco. No SenacGames, as Migrations são a ponte que converteu o modelo C# em instruções `CREATE TABLE` do SQL.

07 SEGURANÇA ASP.NET IDENTITY

A segurança é o pilar fundamental de qualquer aplicação corporativa. Quem é o usuário? Ele tem permissão para apagar um game?

O **ASP.NET Core Identity** é o porteiro do SenacGames. Ele nos poupou o trabalho de criar tabelas de login, hash de senhas e lógicas de recuperação complexas do zero.

Componentes de segurança do SenacGames

- 1 Authentication (Autenticação):** responde "QUEM é você?". Quando o usuário faz login, a API valida o e-mail e a senha.
- 2 Authorization (Autorização):** responde "O QUE você pode fazer?". O usuário autenticado quer apagar um game. Ele é Administrador? Se for Cliente, acesso negado!
- 3 Users e Password Hash:** o Identity nunca salva a senha pura (`"123456"`) no SQL. Ele gera um Hash irreversível. Se o banco vazar, os hackers não descobrirão a senha!
- 4 Roles (Perfis):**
 - **Administrador:** acesso total (cria games, gerencia usuários).
 - **Cliente:** consumidor final.
- 5 Claims (Afirmações):** são pedaços de informação sobre o usuário embutidos no token. Por exemplo: o nome do usuário e o caminho da foto de perfil. O SenacGames utiliza `Claim` para mostrar o nome sem ter que ir ao banco de dados toda vez!
- 6 JWT (JSON Web Tokens):** a API, após confirmar o login, devolve um token JWT (uma credencial digital). O MVC e o Desktop guardam esse token. Em toda requisição futura, eles enviam o JWT no cabeçalho.
- 7 Cookies:** no painel Web (MVC), nós envelopamos esse JWT dentro de um Cookie criptografado. Assim, o navegador gerencia a sessão, garantindo login/logout de forma fluida.

A API (Application Programming Interface) é o cérebro sem rosto do SenacGames. Ela não possui telas, não possui botões e não possui cores. Ela apenas recebe perguntas e responde com dados.

Nossa arquitetura usa a API como centro nervoso porque queremos servir múltiplas plataformas. A Loja Web consome a API. O Aplicativo Desktop consome a **mesma** API. Um futuro App Mobile consumiria a **mesma** API!

O que é uma API REST?

REST é um conjunto de regras (boas práticas) de como conversar via Web. A conversa é baseada no padrão HTTP.

Controllers e Endpoints — o `GamesController.cs` na API possui "Endpoints" (portas lógicas). Cada Endpoint tem uma responsabilidade:

GET `/api/games` — Me dê todos os games (Ler)

GET `/api/games/5` — Me dê o game ID 5

POST `/api/games` — Aqui estão os dados, CRIE um game

PUT `/api/games/5` — Aqui estão os dados, ATUALIZE o game 5

DELETE `/api/games/5` — APAGUE o game 5

Status HTTP

A API usa o vocabulário oficial da Web para responder:

CÓDIGO	SIGNIFICADO
200 OK	Deu tudo certo, aqui estão seus dados.
400 BadRequest	Você me enviou dados inválidos, corrija e tente de novo.
401 Unauthorized	Você precisa fazer login primeiro.
403 Forbidden	Você está logado, mas não tem permissão para isso.
404 NotFound	O game não existe.
500 InternalServerError	Algo explodiu no servidor.

Os dados transitam no formato JSON, uma linguagem universal que qualquer sistema entende:

```
{  
  "id": 1,  
  "title": "SenacGames: O Início",  
  "releaseYear": 2024  
}
```

Swagger

Quando você roda a API no Visual Studio, uma tela preta e verde aparece. É o Swagger! Ele lê o seu código e monta um "Manual de Instruções" da API automaticamente, permitindo testar os endpoints sem precisar programar nada.

Por que existe e qual problema resolve?

Se você usar o `_context.Games.Add()` direto no `Controller` da API, você criou um problema grave: a API está intimamente "casada" com o Banco de Dados. Se um dia a equipe decidir trocar o SQL Server por MongoDB (ou Oracle), você terá que reescrever todos os Controllers.

O **Repository Pattern (Padrão de Repositório)** é um intermediário. Ele diz: "Não fale com o banco, fale comigo! Eu resolvo!".

O fluxo completo

- 1 A API pede os dados ao `Service`.
- 2 O `Service` pede os dados ao `Repository`.
- 3 O `Repository` (e só ele) chama o Entity Framework.

Exemplo no SenacGames

Temos a Interface `IGameRepository` e a classe `GameRepository`:

```
public async Task<Game> GetByIdAsync(int id)
{
    return await _context.Games
        .Include(g => g.Category)
        .FirstOrDefaultAsync(g => g.Id == id);
}
```

A API nem faz ideia de que o EF Core existe! Ela só sabe que chamou `GetByIdAsync` e a mágica aconteceu.

Regras de negócio e validações

Onde colocamos a regra: "o usuário não pode cadastrar um game sem Category"? No Controller? **Não!** O Controller só serve para repassar recados. No Repository? **Não!** O Repository só faz leitura/escrita no banco.

A lógica de negócios vai no **Service Pattern**! O Service é o cérebro da operação.

Organização

Toda validação e orquestração fica aqui. Se eu preciso deletar um game, o Service faz o seguinte:

- 1 Pede ao Repository para buscar o game.
- 2 Verifica se o game existe (se não, lança erro).
- 3 Pede ao Repository para deletar o game.

Exemplo

```
public async Task AddAsync(GameDto dto)
{
    if (string.IsNullOrEmpty(dto.Title))
        throw new ArgumentException("O título é obrigatório");

    var game = new Game { Title = dto.Title };
    await _gameRepository.AddAsync(game);
}
```


DTOS (DATA TRANSFER OBJECTS)

Por que não enviar as "Entities" diretamente?

Imagine a classe `ApplicationUser`. Ela tem a propriedade `PasswordHash` (a senha criptografada). Se a API retornar a classe `ApplicationUser` inteira num `GET /api/users`, o Frontend receberá o Hash da Senha de todos os usuários! Isso é um **grave vazamento de dados de segurança**!

O mapeamento (a solução)

Criamos um **DTO (Objeto de Transferência de Dados)**. O DTO é uma classe "burra" e enxuta, feita sob medida para a tela. Exemplo `AuthDto`:

```
public class AuthDto
{
    public string Email { get; set; }
    public string Password { get; set; }
}
```

Boas práticas

- Entradas (`POST` / `PUT`) recebem DTOs (ex: `GameDto`).
- Saídas (`GET`) retornam DTOs (ex: `GameDto`).
- Apenas as camadas internas (Domain, Infra, Service) conhecem as Entidades reais. A API serve apenas os DTOs.

Se a API não pode retornar Entidades, e sim DTOs, como copiamos os dados de um para o outro? A forma "braçal" é fazer isso:

```
var dto = new GameDto();
dto.Title = game.Title;
dto.ReleaseYear = game.ReleaseYear;
// ... e fazer isso para várias propriedades!
```

O funcionamento

O **AutoMapper** (ou Mapster, ou outro mapeador automático) é uma biblioteca que faz isso magicamente. Você cria um Perfil de Mapeamento:

```
CreateMap<Game, GameDto>();
```

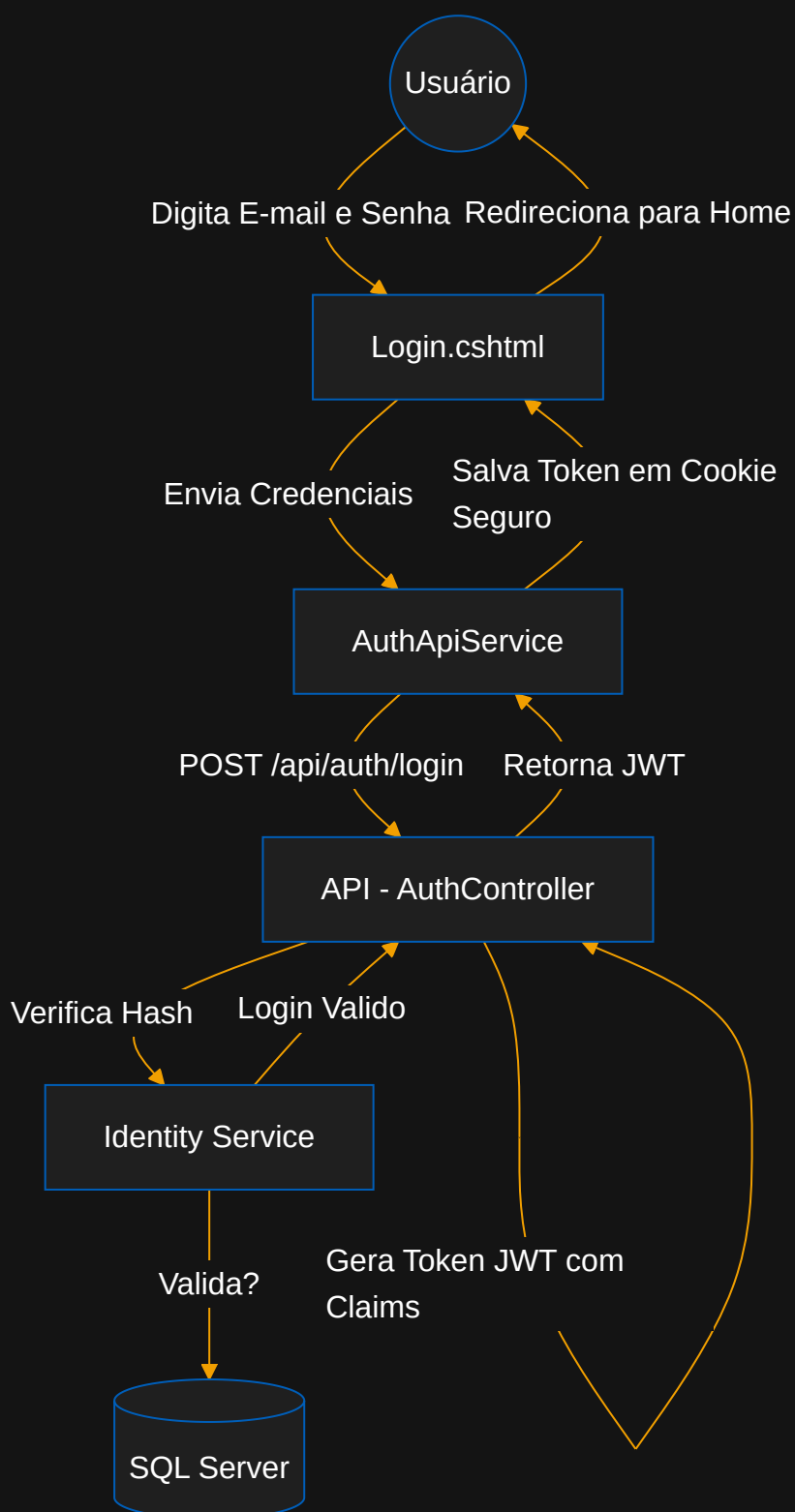
E então, no código, basta pedir ao Mapper para traduzir:

```
var dto = _mapper.Map<GameDto>(game);
```

O AutoMapper olha as propriedades com o mesmo nome e tipo (ex: `Title` no Game e `Title` no DTO) e copia tudo sozinho, poupando milhares de linhas de código repetitivo e prevenindo erros humanos ("esqueci de copiar o Id!").

O login não é apenas verificar se a senha está certa. É gerar uma permissão que durará horas.

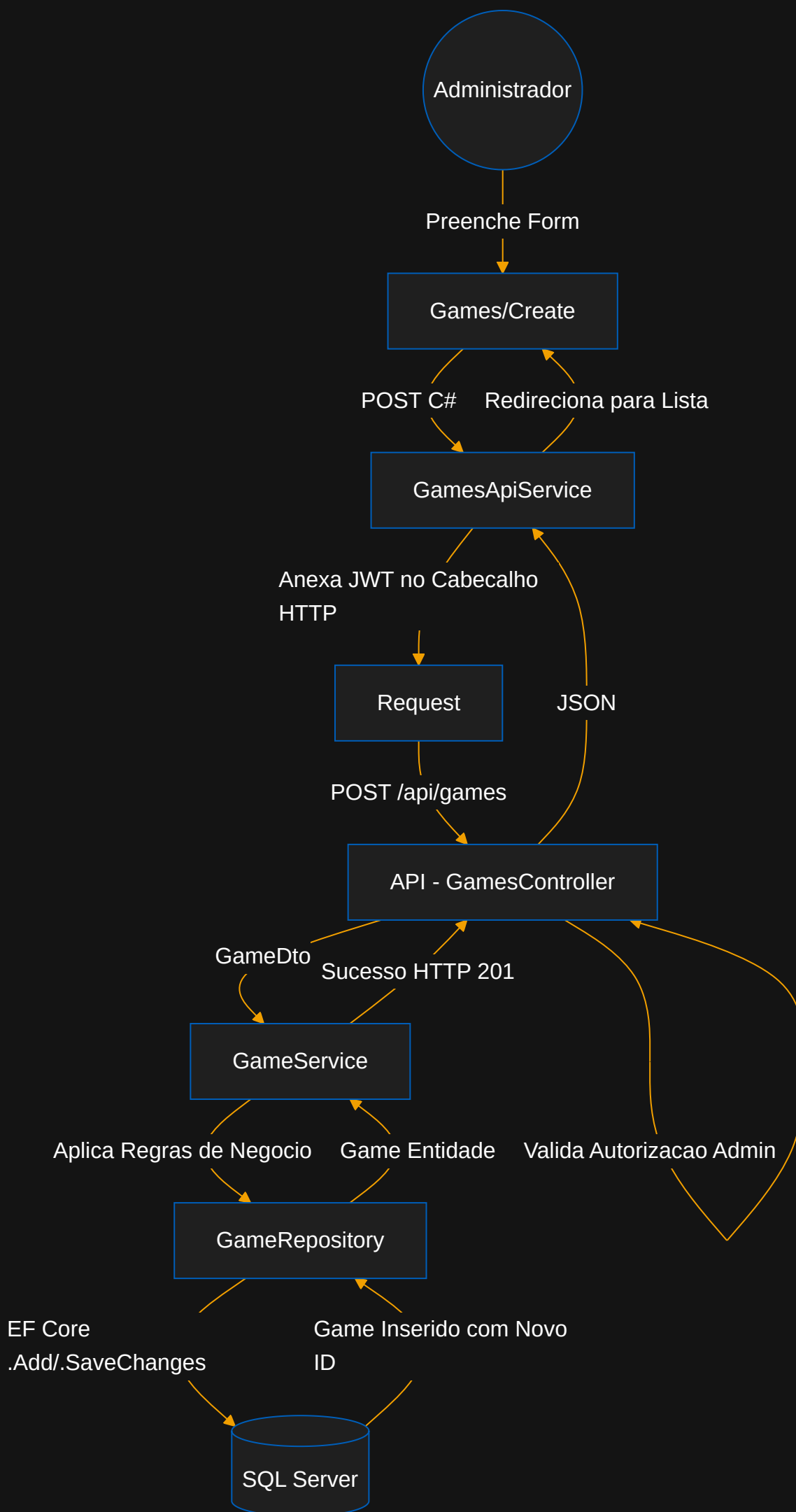
O diagrama completo



Quando você clica em "Entrar", a MVC pede à API para validar as credenciais. A API invoca o Identity, que consulta o SQL. A senha está criptografada (hash), então o Identity recriptografa o que você digitou e compara as *hashes*. Estando tudo certo, a API assina um **JWT** (Token de acesso) e manda de volta para a Web MVC. A Web MVC guarda esse token num Cookie para que o navegador se lembre de quem você é a cada clique nas páginas.

FLUXO DO CADASTRO DE GAMES

O administrador deseja adicionar um novo game no sistema. Veja o longo, porém organizado, caminho percorrido:



Este fluxo demonstra a segurança profunda: mesmo que um "hacker" tente enviar o comando via Postman, a etapa de **Valida Autorização** barrará a execução caso o JWT não contenha o perfil "Administrador".

Para manter a sanidade mental de uma equipe de programação, o código precisa ter um "lugar para cada coisa, e cada coisa em seu lugar". No SenacGames temos a seguinte estrutura base:

```
SenacGames/
├── SenacGames.Domain/
│   └── Entities (Game.cs, Category.cs)
├── SenacGames.Infrastructure/
│   ├── Context/ (SenacGamesDbContext.cs)
│   ├── Migrations/ (Arquivos gerados pelo EF Core)
│   └── Repositories/ (Implementações do banco, ex: GameRepository.cs)
├── SenacGames.Application/
│   ├── DTOs/ (GameDto.cs, AuthDto.cs)
│   └── Services/ (GameService.cs, CategoryService.cs)
├── SenacGames.API/
│   ├── Controllers/ (GamesController.cs, AuthController.cs)
│   └── Program.cs (Onde ligamos todas as Injeções de Dependência)
├── SenacGames.UI/
│   ├── Controllers/ (Painel Web - GamesController.cs)
│   ├── Views/ (HTML - Home/Index.cshtml)
│   └── wwwroot/ (CSS, JS, Imagens, Uploads)
├── SenacGames.Desktop/
│   ├── Forms/ (Telas WinForms)
│   └── Services/ (Classes que consomem a API, ex: GamesApiService.cs)
└── Docs/
    └── Documentação oficial (este arquivo inclusive)
```

- `wwwroot`: essa pasta no projeto MVC é especial. Ela é a **única** pasta que o navegador do usuário final consegue acessar (onde ficam as imagens e folhas de estilo CSS públicas).
- `Program.cs`: é o "botão de ligar" do sistema. Lá nós configuramos o Banco de Dados, o Identity, o Swagger e registramos nossos Services.

Lembra de quando você fazia Banco de Dados no primeiro módulo e tinha que guardar um arquivo `script.sql` para rodar na máquina do colega? As **Migrations** do Entity Framework eliminam esse trabalho manual. Elas são o "Controle de Versão" do Banco de Dados.

1. `Add-Migration NomeDaMudanca`

Quando você cria uma classe nova ou altera uma propriedade, você abre o "Package Manager Console" e digita esse comando. O EF vai comparar as suas Classes com o Banco atual e vai gerar um arquivo C# na pasta `Migrations` com as instruções (UP: "Crie a coluna" e DOWN: "Remova a coluna").

2. `Update-Database`

Este comando pega todas as Migrations que o seu banco local ainda não tem e aplica-as fisicamente no SQL Server. Ele envia os `ALTER TABLE` e `CREATE TABLE` !

O código funciona, mas ele tem qualidade? O SenacGames utiliza boas práticas globais.

1. Injeção de Dependência (DI)

Em vez de instanciar classes com `new GameRepository()`, nós injetamos.

```
public class GamesController
{
    private readonly IGameService _service;
    public GamesController(IGameService service) { _service = service; }
}
```

Isso permite que a configuração no `Program.cs` passe a instância certa e ajuda a criar Testes de Software.

2. SOLID (S = Single Responsibility Principle)

Cada classe deve ter apenas um motivo para mudar.

- O Controller só liga para rotas (MVC).
- O Service só liga para regras de negócios.
- O Repository só liga para o Banco.

Se der erro no banco, você sabe exatamente qual classe abriu (e não precisa caçar no meio do HTML).

3. Clean Code (Código Limpo)

Utilizamos nomes em inglês técnicos (`GetByIdAsync`, `GamesController`) para padronizar com as ferramentas globais. Métodos curtos, classes bem definidas e separadas em pastas categóricas.

4. Retornos Assíncronos (`async` / `await`)

Toda chamada que envolve internet, leitura de disco ou banco de dados no SenacGames utiliza `Task` e `async` / `await`. Isso significa que o servidor (IIS/Kestrel) não vai "travar" a thread enquanto espera a resposta do Banco de Dados. O sistema fica milhares de vezes mais rápido para atender dezenas de usuários simultâneos!

? Por que usar Repository Pattern e não colocar o `_context` na API direto?

Para manter a sua API limpa e independente. Se amanhã o sistema trocar o tipo de banco de dados, você troca apenas o Repository, e toda a API e UI continuam funcionando intactas. Além disso, permite Testes Unitários de forma mais fácil.

? Por que usar DTOs em vez de retornar as classes originais do EF?

Por segurança! A classe do EF possui amarras do banco e dados sensíveis. O DTO é uma caixa de transporte apenas com o estritamente necessário.

? Quando utilizar SQL puro?

O EF Core é excelente para 95% do sistema (CRUD, buscas, relatórios simples). Para consultas de milhões de registros, geração de planilhas complexas com diversos JOINS pesados, usar uma `Stored Procedure` em SQL puro pode ser mais otimizado.

? Por que usar Identity? Não posso apenas salvar senha e logar?

O Identity faz o hash moderno das senhas (nunca se salva senha crua!), cuida da expiração do JWT, protege contra ataques básicos, trata os perfis (Roles) de forma padronizada para toda a plataforma Microsoft, e gerencia tokens de recuperação. Construir isso do zero hoje é correr um risco extremo de segurança.

19

PARA PRATICAR

GLOSSÁRIO TÉCNICO

ORM (Object-Relational Mapping)

Ferramenta (como o EF Core) que traduz código C# em SQL e vice-versa.

Migration

Arquivo de código que registra as mudanças (criações/alterações) do Banco de Dados para que ele evolua junto com as classes C#.

Claim

Um pedaço de informação de identidade "costurado" no passaporte (token JWT) do usuário.

Hash

Transformação matemática irreversível de uma string. Uma senha `"123"` vira um Hash gigantesco. Nunca mais se reverte, só se compara.

JWT (JSON Web Token)

Um token de sessão em formato JSON, assinado digitalmente, que prova que o usuário logou recentemente. O usuário envia no "Header" da requisição.

Middleware

São "tubos" pelos quais a requisição passa antes de chegar na API (ex: o Middleware de Autorização intercepta a requisição e expulsa quem não tem o JWT).

Endpoint

Uma URL da sua API disponível para chamadas (ex: `GET /api/games/1`).

LINQ (Language Integrated Query)

Sintaxe C# super poderosa para filtrar dados em Listas (`.Where(x => ...)`) que o EF converte diretamente em linguagem SQL.

Lambda (Expressão Lambda)

Uma função "anônima" e enxuta, escrita com o operador `=>` ("vai para"). Em `g => g.Title = "HaLo"`, o `g` é o parâmetro de entrada e o que vem depois do `=>` é o que a função retorna. É a base do LINQ no SenacGames.

Interface

Um "contrato" que define quais métodos uma classe deve ter, sem dizer como implementá-los (ex: `IGameRepository`). Permite trocar a implementação real sem afetar quem a utiliza.

Injeção de Dependência (DI)

Técnica em que uma classe recebe (em vez de criar) os objetos de que precisa, geralmente pelo construtor. É o que permite ao SenacGames trocar peças do sistema com facilidade.

Async/Await

Palavras-chave do C# que permitem executar operações demoradas (como acesso ao banco) sem travar a aplicação enquanto se espera a resposta.

Razor

Motor de visualização da Microsoft que permite misturar C# (`@foreach` , `@if`) com código HTML.

Seed (Semente)

Injeção de dados automáticos durante a primeira inicialização (carga inicial) do banco de dados para evitar que ele inicie vazio.



Fim de Sessão

Parabéns por concluir o Guia do SenacGames! Estudar essa aplicação te colocará vários passos à frente em arquiteturas modernas.

Mãos à obra e sucesso!